

Source-to-Source Compilation for Hardware/Software Codesign

Tiago Santos @ GNU Tools Cauldron 2025



I. Who are we?



Special-Purpose Computing Systems, Languages and Tools

Objectives

Systematically address non-functional requirements (e.g., execution time, energy)

Research integrated hardware-software solutions

Architectures

From embedded (ARM, RISC-V, FPGAs)...

...to HPC (Intel Xeon, GPUs, FPGAs)

Tools

A family of source-to-source compilers

Binary Translation Framework

Previous work on compiler optimization

Given an application, select the best compiler optimizations to improve execution time and/or energy consumption

- GCC Flag Selection
 - select the set of GCC flags
 - improvements over typical GCC `-Ox` compiler options
 - interaction with GCC via command line
- LLVM phase selection and phase ordering
 - select the set of LLVM phases and the sequence of phases
 - improvements over typical LLVM `-Ox` compiler options
 - interaction with LLVM via `opt`
 - 1.16x to 1.25x (LEON processor, LLVM 3.7)

Source-to-Source Compiler - Example

```
int foo()  
{  
    int a = 0;  
  
    a+= bar1(10);  
  
    a+= bar2(20);  
  
    return a;  
}
```

Language: C

Source-to-Source Compiler - Example

```
int foo()  
{  
    int a = 0;  
  
    a+= bar1(10);  
  
    a+= bar2(20);  
  
    return a;  
}
```

Language: C



```
int foo()  
{  
    int a = 0;  
  
    a+= bar1(10);  
    printf("foo->bar1\n");  
  
    a+= bar2(20);  
    printf("foo->bar2\n");  
  
    return a;  
}
```

Language: C

Why source-to-source (S2S)?

When researching using traditional compilers we may...

- ...lose significant information (e.g. comments, high-level structures, loops)
- ...have a high-learning curve
- ...be impractical to distribute
- ...be locked to a specific compiler version
- ...impose vendor lock-in (aggravated in fragmented environments, e.g. embedded)

Why source-to-source (S2S)?

When researching using traditional compilers we may...

- ...lose significant information (e.g. comments, high-level structures, loops)
S2S: keep all source code information
- ...have a high-learning curve
- ...be impractical to distribute
- ...be locked to a specific compiler version
- ...impose vendor lock-in (aggravated in fragmented environments, e.g. embedded)

Why source-to-source (S2S)?

When researching using traditional compilers we may...

- ...lose significant information (e.g. comments, high-level structures, loops)

S2S: keep all source code information

- ...have a high-learning curve

S2S: use DSLs/APIs over AST or similar structures

- ...be impractical to distribute

- ...be locked to a specific compiler version

- ...impose vendor lock-in (aggravated in fragmented environments, e.g. embedded)

Why source-to-source (S2S)?

When researching using traditional compilers we may...

- ...lose significant information (e.g. comments, high-level structures, loops)

S2S: keep all source code information

- ...have a high-learning curve

S2S: use DSLs/APIs over AST or similar structures

- ...be impractical to distribute

S2S: just another tool in the tool-chain

- ...be locked to a specific compiler version

- ...impose vendor lock-in (aggravated in fragmented environments, e.g. embedded)

Why source-to-source (S2S)?

When researching using traditional compilers we may...

- ...lose significant information (e.g. comments, high-level structures, loops)

S2S: keep all source code information

- ...have a high-learning curve

S2S: use DSLs/APIs over AST or similar structures

- ...be impractical to distribute

S2S: just another tool in the tool-chain

- ...be locked to a specific compiler version

S2S: source-code is the interface

- ...impose vendor lock-in (aggravated in fragmented environments, e.g. embedded)

Why source-to-source (S2S)?

When researching using traditional compilers we may...

- ...lose significant information (e.g. comments, high-level structures, loops)

S2S: keep all source code information

- ...have a high-learning curve

S2S: use DSLs/APIs over AST or similar structures

- ...be impractical to distribute

S2S: just another tool in the tool-chain

- ...be locked to a specific compiler version

S2S: source-code is the interface

- ...impose vendor lock-in (aggravated in fragmented environments, e.g. embedded)

S2S: compatible with any compiler that accepts the language, preserving retargetability

Why source-to-source (S2S)?

Other points

- Easier retargeting of different tools and target machines
 - Users/developers can access source code with the transformations applied
- ~
- Easier integration of APIs and libraries (e.g., the ones for offloading computations to hardware accelerators)

We see S2S as complementary to traditional compiler technologies (e.g., prototyping)

Our family of compilers



Java to Java



C/C++ to C/C++

...plus a few nascent tools:

- Alpakka for smali (Android bytecode, directly supports APKs)
- Metafor for Fortran (uses LLVM Flang as parser)
- AnyWeaver, which accepts ANTLR grammars

One common framework

High productivity



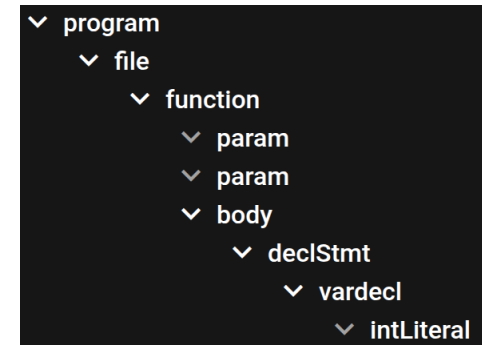
Write transformations in modern
JIT-compiled languages
(JavaScript or TypeScript).
Node.js is the main orchestrator

High composability



Easily reuse, distribute and
combine analysis and
transformations through NPM

Don't reinvent the wheel



We leverage existing state-of-the-art
parsers and focus on providing APIs
to analyze and modify the AST

Clava: C/C++ to C/C++



Extensive support of C and C++ constructs

Leverages existing parser from clang to:

- Generate the AST
- Verify if transformations over the AST are syntactically correct
- Allow for the transpilation of code written under any C or C++ standard

Active extension development

8+ distinct extensions developed and published in the last 1.5 years:

- A collection of general-purpose code transformations
- AST visualization and debug tools
- Code compliance verification and correction
- Multiple graph-based representations of an application
- Integration with downstream vendor tools
- ...and more

CMake integration



Easily integrate Clava as part of a CMake-based workflow

Extending Clava using JavaScript

Log after every call the name of the current function and the name of the call being called

```
import Query from "@specs-feup/lara/api/weaver/Query.js";
```

Language: JS

Extending Clava using JavaScript

Log after every call the name of the current function and the name of the call being called

```
import Query from "@specs-feup/lara/api/weaver/Query.js";  
import {FunctionJp, Call} from "@specs-feup/clava/api/Joinpoints.js";
```

Language: JS

Extending Clava using JavaScript

Log after every call the name of the current function and the name of the call being called

```
import Query from "@specs-feup/lara/api/weaver/Query.js";
import {FunctionJp, Call} from "@specs-feup/clava/api/Joinpoints.js";

for(const func of Query.search(FunctionJp)) {

}
```

Language: JS

Extending Clava using JavaScript

Log after every call the name of the current function and the name of the call being called

```
import Query from "@specs-feup/lara/api/weaver/Query.js";
import {FunctionJp, Call} from "@specs-feup/clava/api/Joinpoints.js";

for(const func of Query.search(FunctionJp)) {
    for(const call of Query.searchFrom(func, Call)) {

    }
}
```

Language: JS

Extending Clava using JavaScript

Log after every call the name of the current function and the name of the call being called



```
import Query from "@specs-feup/lara/api/weaver/Query.js";
import {FunctionJp, Call} from "@specs-feup/clava/api/Joinpoints.js";

for(const func of Query.search(FunctionJp)) {
    for(const call of Query.searchFrom(func, Call)) {
        call.insertAfter(`printf("${func.name}->${call.name}\\n");`);
    }
}
```

Language: JS

CMake Example

```
cmake_minimum_required (VERSION 3.31)
project (HELLO)

add_executable (helloDemo src/test.c)
```

Language: CMake

CMake Example

```
cmake_minimum_required (VERSION 3.31)
project (HELLO)

add_executable (helloDemo src/test.c)

find_package (Clava REQUIRED)
```

Language: CMake

CMake Example

```
cmake_minimum_required (VERSION 3.31)
project (HELLO)

add_executable (helloDemo src/test.c)

find_package (Clava REQUIRED)
clava_weave(helloDemo clava_script.js)
```

Language: CMake

Recent work

AutoPar: Automatic Parallelisation of for-loops

- Statically analyses and inserts OpenMP pragmas
- Automatic, no user effort
- Developed by Hamid Arabnejad, post-doc researcher
- Experiments
 - NAS – 2 ×
 - PolyBench – 8.6 × (8 threads, XL)
 - Himeno – 10 × (16 threads)

```
#pragma omp parallel for default(shared) firstprivate(numIter) reduction(+ : a)
for (int i = 0; i < numIter; i++) {
    a += i;
}
```

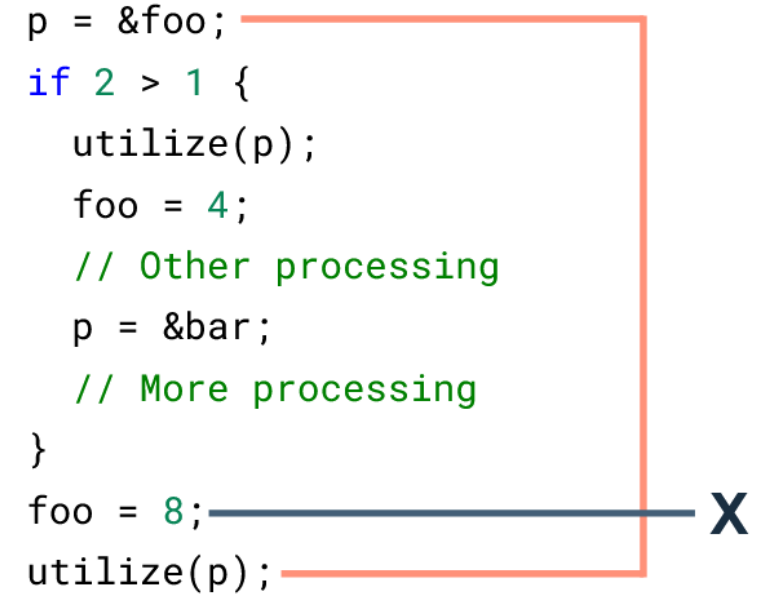
Language: C

Recent work

CORAL: a Rust-like Borrow Checker for C

- C is an unsafe language
- Replicate Rust's memory safety guarantees in C
- Static analysis before passing code to compiler:
 - Use while Borrowed Detection
 - Multiple Indirection and Reborrow
 - Lifetime Elision
 - ... more coming
- Developed by Tiago Duarte da Silva and Pedro Gonalo Correia, MSc students

```
p = &foo;
if 2 > 1 {
    utilize(p);
    foo = 4;
    // Other processing
    p = &bar;
    // More processing
}
foo = 8;
utilize(p);
```



```
error[E0506]: Cannot write to 'foo' while borrowed
----> nll_used_while_borrowed.c:18

10 |     p = &foo;
    |         (shared) borrow of 'foo' occurs here
18 |     foo = 8;
    |         write to 'foo' occurs here, while borrow is still active
19 |     use(p);
    |         borrow is later used here
```

Work and figures by Tiago Duarte da Silva

Recent work

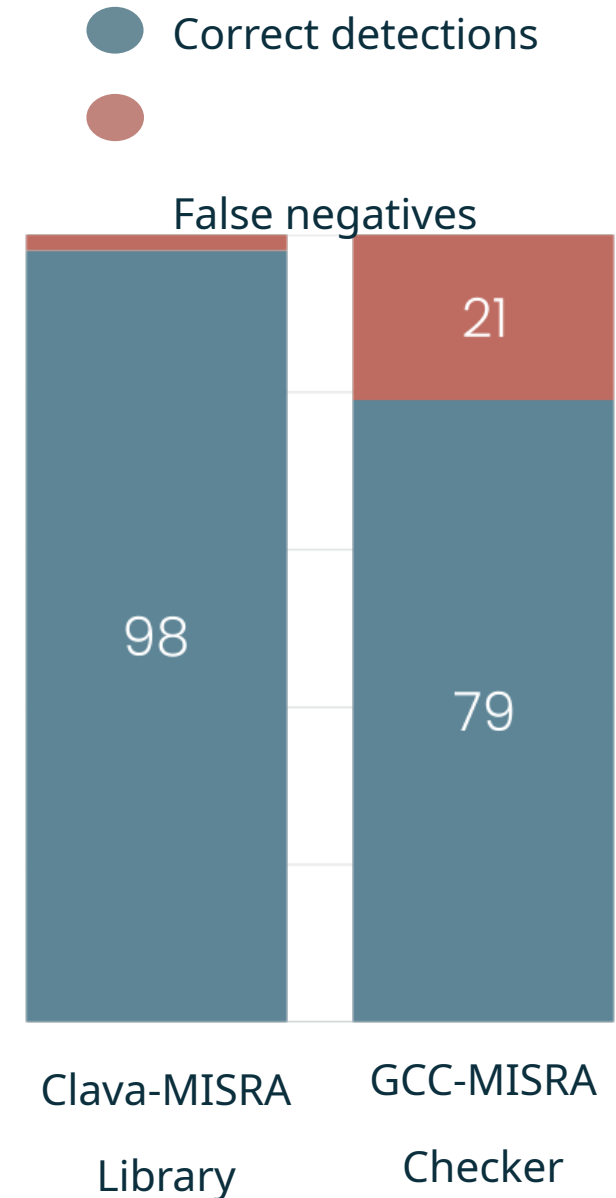
Source-to-source approach for MISRA-C

- C is used in critical systems (aviation, automotive) with coding standards such as MISRA-C
- Improve analysis, automate correction
- Developed by Milena Gouveia, MSc student

```
sizeof(--i); // '--i' has side effects  
sizeof(incr()); // function call has side effects  
sizeof(int[j++]); // VLA type with side effects
```



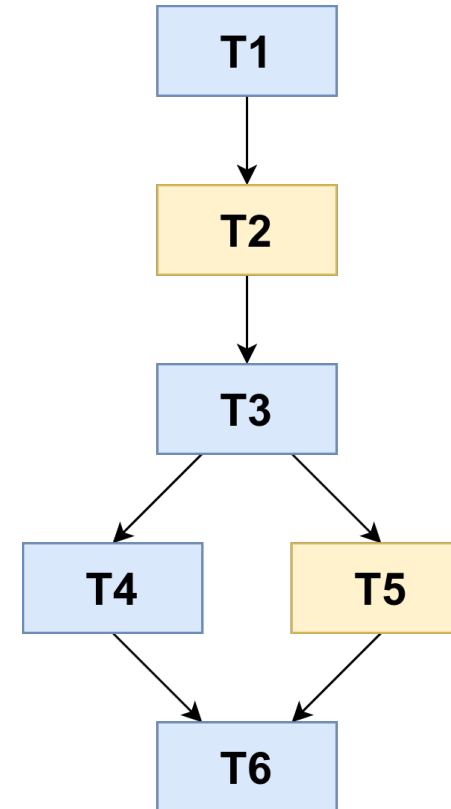
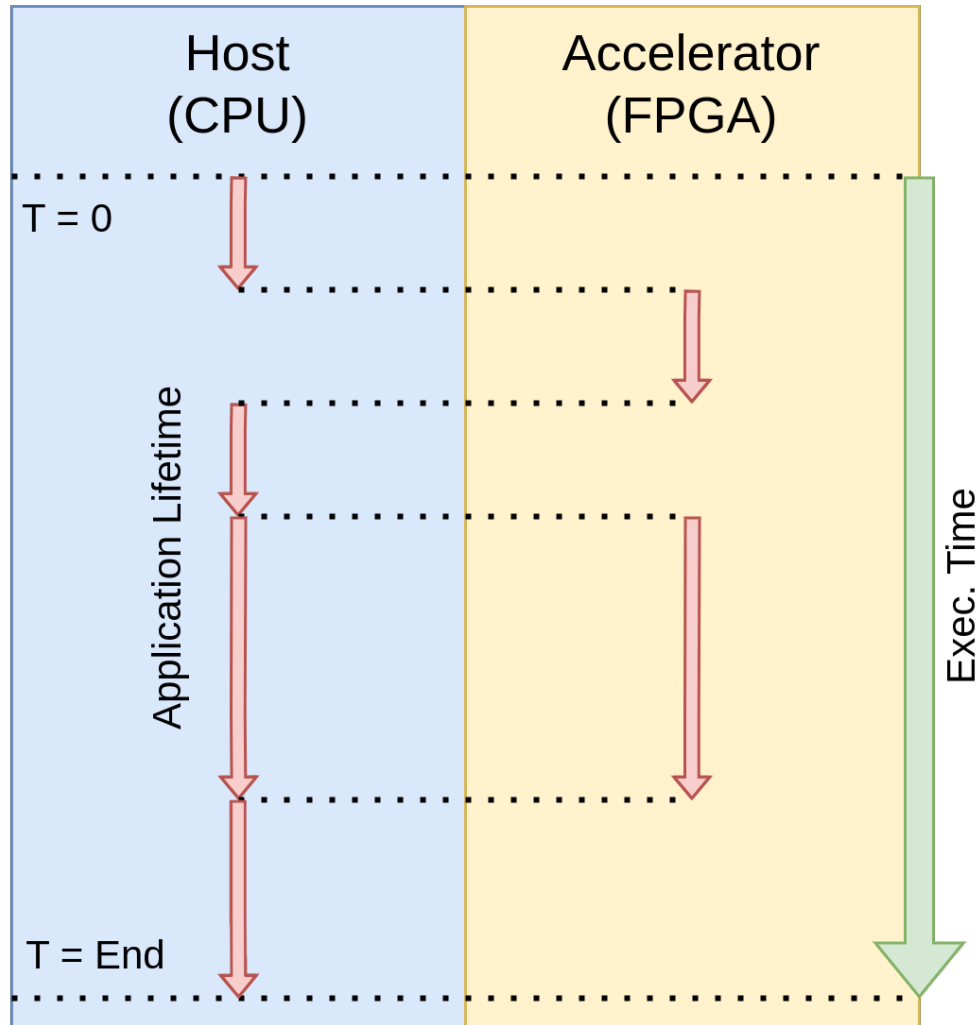
```
sizeof(i); // compliant  
sizeof(int); // compliant  
sizeof(int[j++]); // VLA type with side effects
```





II. Hardware/Software Co-design

Hardware acceleration of an application



How do we automatically find, select and optimize the regions to be offloaded?

Source-to-source can help!

```
clock_t start = clock();
dotproduct(A, B);
clock_t end = clock();
double time = ((double)(end-start)) / CLOCKS_PER_SEC;
printf("%f\n", time);
```

```
...
auto A_buf = xrt::bo(device, N * sizeof(int),
                    kernel.group_id(0));
auto B_buf = xrt::bo(device, N * sizeof(int),
                    kernel.group_id(1));

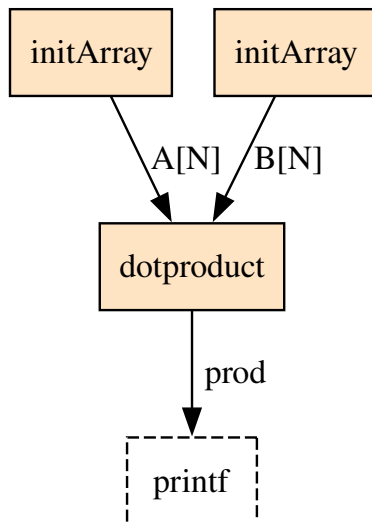
A_buf.write(A.data());
B_buf.write(B.data());

A_buf.sync(XCL_BO_SYNC_BO_TO_DEVICE);
B_buf.sync(XCL_BO_SYNC_BO_TO_DEVICE);

auto run = kernel(A_buf, B_buf);
run.wait();
...
```

```
int dotproduct(int *A, int *B)
{
    int prod = 0;
    for (int i = 0; i < N; i++)
        prod += A[i] * B[i];
}
```

```
int dotproduct(int A[N], int B[N]) {
    #pragma hls array_partition variable=A cyclic factor=32
    #pragma hls array_partition variable=B cyclic factor=32
    int prod = 0;
    for (int i = 0; i < N; i++) {
        #pragma hls unroll factor=32
        prod += A[i] * B[i];
    }
}
```



Code transformations

```
void foo(int *A) {  
    int x = 0;  
    #pragma clava begin_outline  
    int y = 5;  
    for (int i = 0; i < 100; i++) {  
        A[i] = x + y * i;  
    }  
    #pragma clava end_outline  
    A[0] = i;  
}
```



```
void outlined_region(int *A, int x, int *i)  
{  
    int y = 5;  
    for ((*i) = 0; (*i) < 100; (*i)++) {  
        A[i] = x + y * (*i);  
    }  
}  
  
void foo(int *A) {  
    int x = 0;  
    int i;  
    outlined_fun(A, x, &i);  
    A[0] = i;  
}
```

Function outlining

Code transformations

```
typedef struct {
    float x;
    float y;
} Point2D;

void foo(Point2D point) { /*...*/ }

void bar(Point2D *point) { /*...*/ }

int main() {
    Point2D point;
    Point2D points[2] = {1.0, 2.0, 3.0, 4.0};

    point.x = 40.0;
    points[1].y = 50.0

    foo(point);
    bar(&point);
}
```



```
void foo(float point_x, float point_y)
{...}

void bar(float *point_x, float *point_y)
{...}

int main() {
    float point_x = 1.0;
    float point_y = 2.0;
    float point_x[2] = {1.0, 3.0};
    float point_y[2] = {2.0, 4.0};

    point_x = 40.0;
    points_y[1] = 50.0;

    foo(point_x, point_y);
    bar(&point_x, &point_y);
}
```

Struct flattening

Code transformations

main.c

```
void foo(int *A);
int bar(*A);
external int globalVar;

int main() {
    int A[1024];
    foo(A);
    int res = bar(A);
    res += globalVar;
    printf("%d\n", res);
    return 0;
}
```

foo.c

```
external int globalVar;

void foo(int *A) {
    int x = globalVar;
    //...
}
```

bar.c

```
int globalVar = 10;

int bar(int *A) {
    //...
}
```

amalgamation.c

```
int globalVar = 10;

void foo(int *A) {
    int x = globalVar;
    //...
}

int bar(int *A) {
    //...
}

int main() {
    int A[1024];
    foo(A);
    int res = bar(A);
    res += globalVar;
    printf("%d\n", res);
    return 0;
}
```



File amalgamation

...and many more, including array flattening, function voidification, call hoisting, and constant folding/propagation

Live demo: Code transformations

Generating Extended Task Graphs

```
void edge_detect(int image_rgb[H][W * 3],
                 int image_gray[H][W],
                 int temp_buf[H][W],
                 int filter[K][K],
                 int output[H][W])
{
    rgbToGrayscale(image_rgb, image_gray);

    filter[0][0] = 1;
    //...
    filter[2][2] = 1;

    convolve2d(image_gray, filter, output);

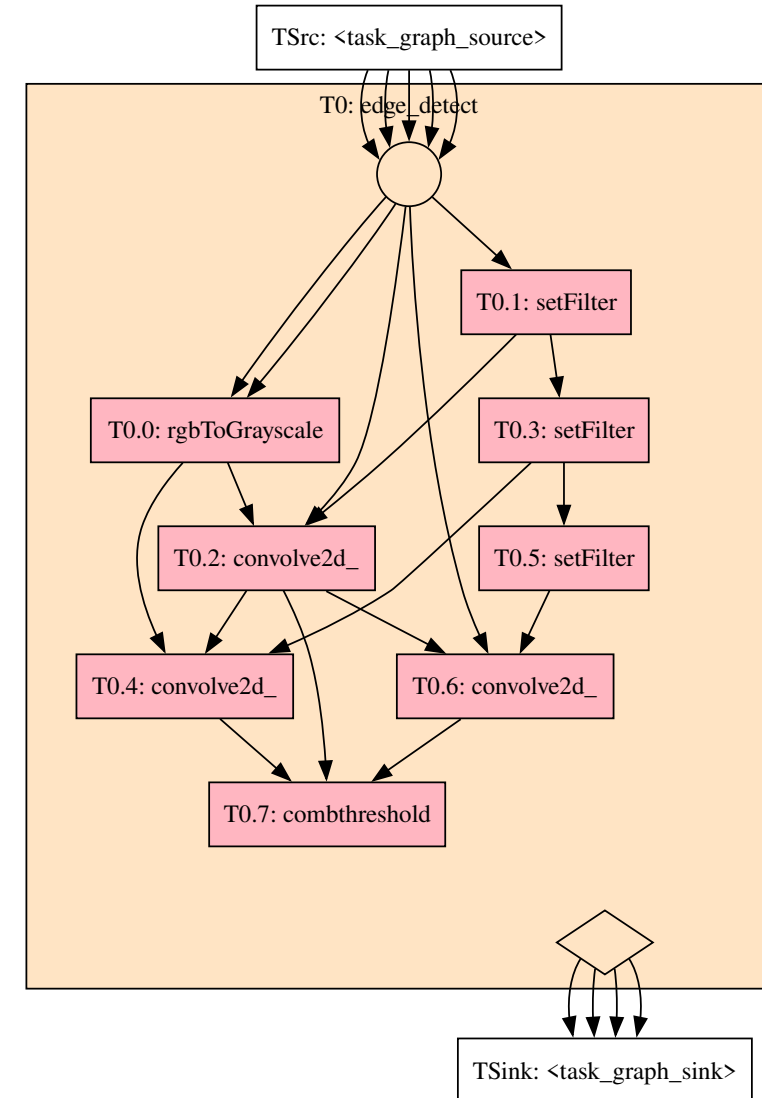
    filter[0][0] = 1;
    //...
    filter[2][2] = -1;

    convolve2d(output, filter, image_gray);

    filter[0][0] = 1;
    //...
    filter[2][2] = -1;

    convolve2d(output, filter, temp_buf);

    combthreshold(image_gray, temp_buf, output);
}
```



What if there are no clear-cut tasks?

Type A: functions with **only** computations

```
void foo(int *A, int *B) {  
    A[0] = 5;  
  
    for (int i = 1; i < N; i++) {  
        A[i] = A[0] * B[i] + A[i];  
    }  
  
    B[0] = N * A[0] + B[0];  
}
```

Type B: functions with **only** function calls and shallow control flow

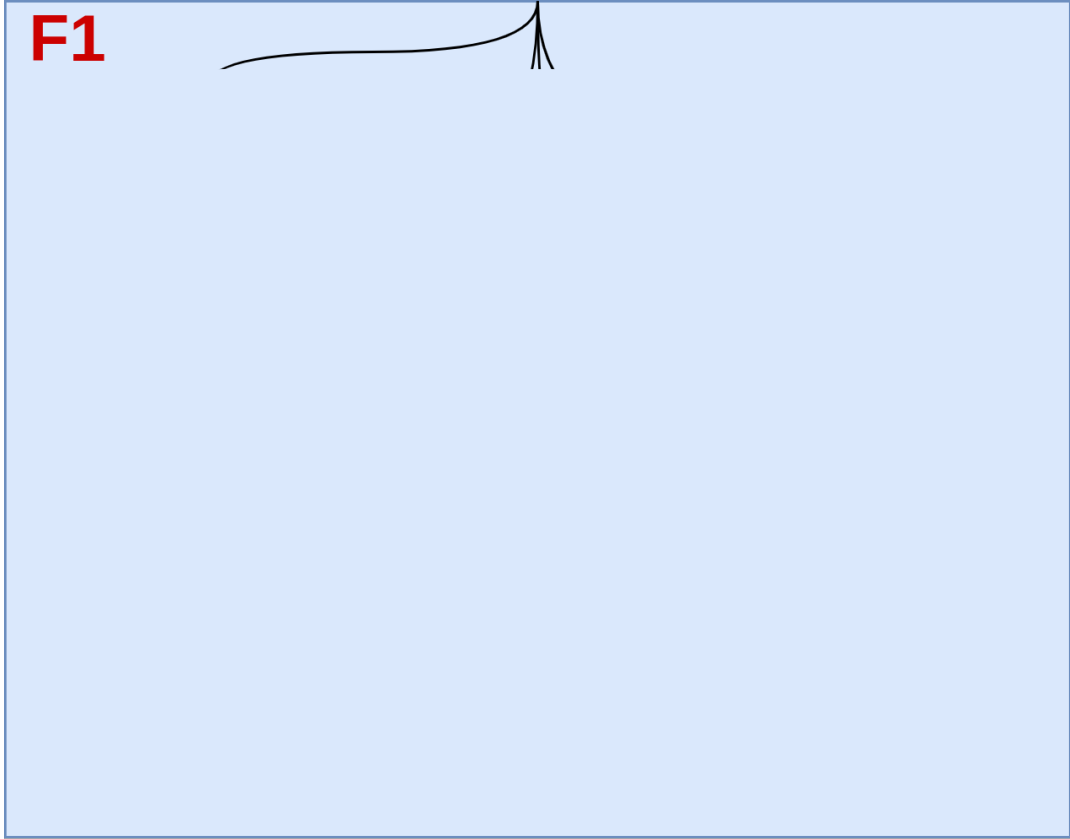
```
void bar(int *A, int *B, bool c) {  
    foo(A, B);  
  
    if (c) {  
        fizz(A);  
    }  
  
    for (int i = 0; i < 10; i++) {  
        foo(B, A);  
        fizz(B);  
    }  
}
```

We apply code transformations to bring an application down to this subset!

An ETG is an overlay of the AST

```
void F1(int *A, int *B, int *C, int *D) {
```

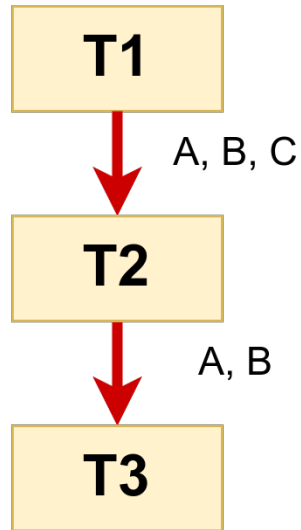
F1



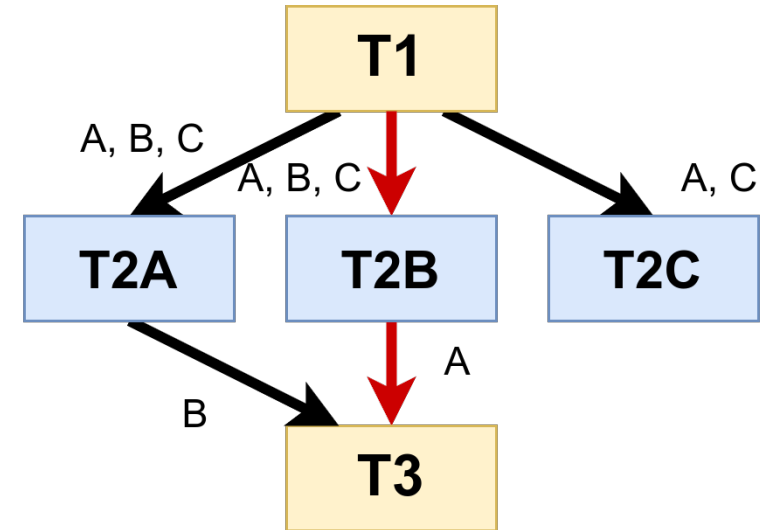
```
}
```

AST Transformation	ETG Transformation
Function Inlining	Merging Tasks
Function Outlining	Splitting a Task

Changing granularity



Increasing Granularity of T2
(Graph-space transformation)



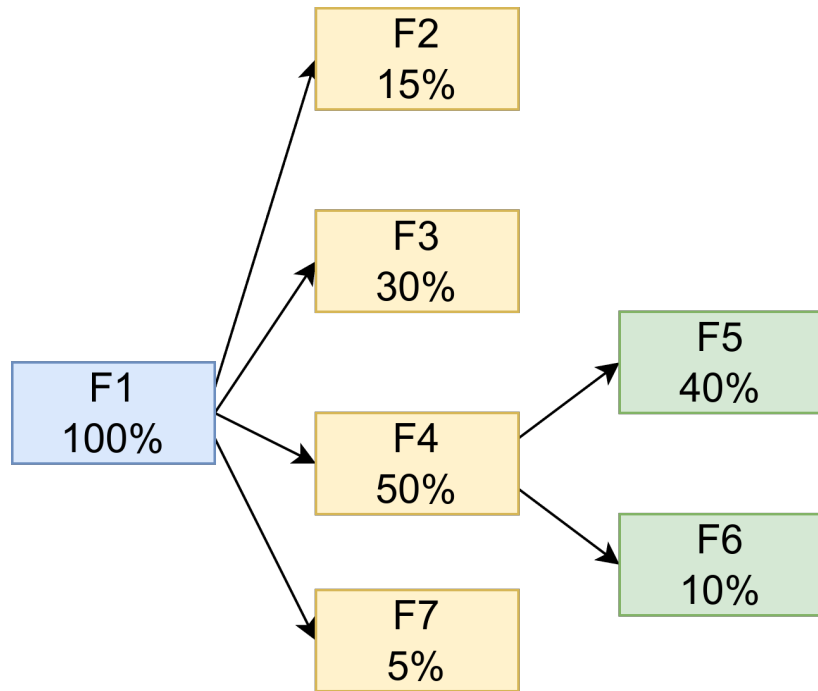
```
void T2(int *A, int *B, int *C) {  
    A[0] = B[1] + C[2];  
    B[0] = A[1] + C[3];  
    C[0] = A[1] + A[1];  
}
```



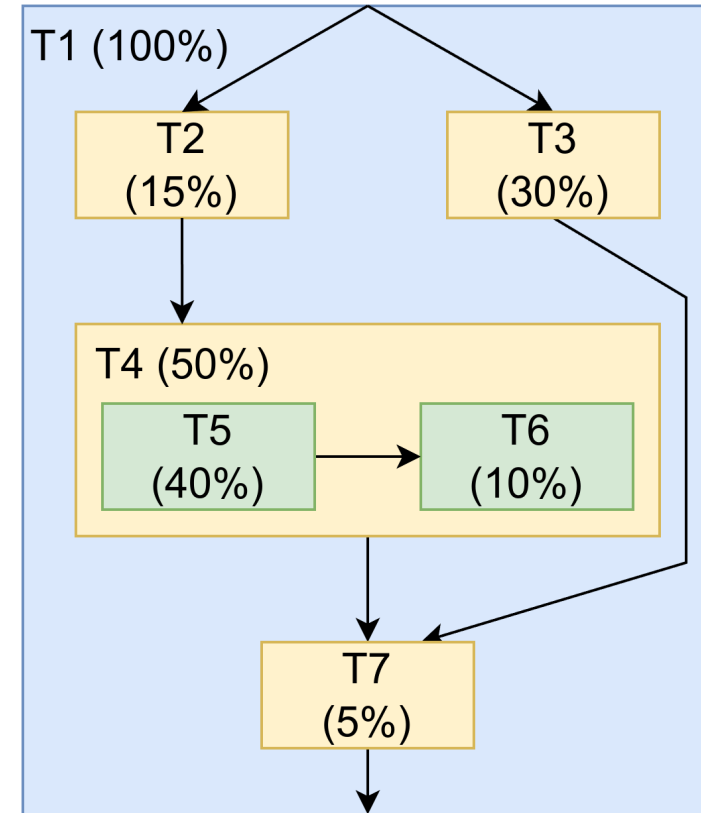
Function outlining
(AST-space transformation)

```
void T2A(int *A, int *B, int *C) {  
    A[0] = B[1] + C[2];  
}  
  
void T2B(int *A, int *B, int *C) {  
    B[0] = A[1] + C[3];  
}  
  
void T2C(int *A, int *C) {  
    C[0] = A[1] + A[1];  
}
```

Annotating an ETG



Call graph with % of execution time spent on each function, obtained using perf or gprof

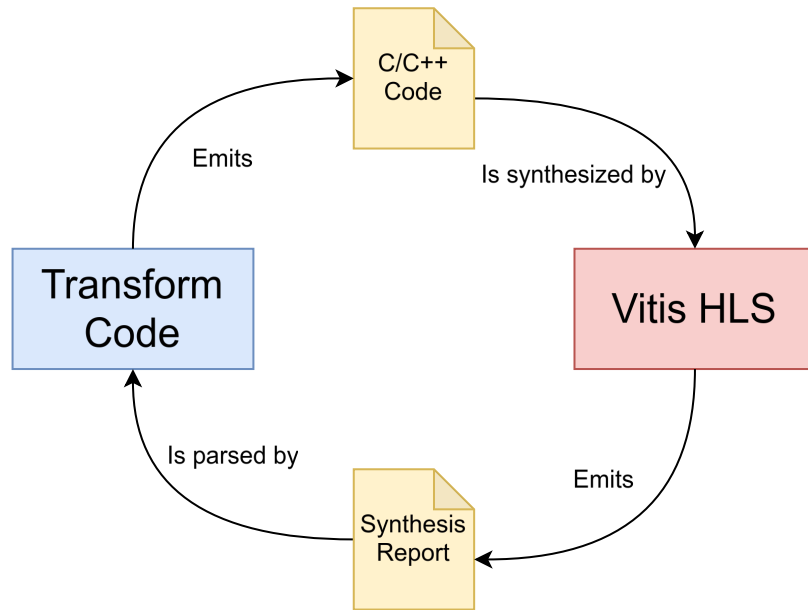


ETG representation annotated with profiling results

Live demo: Generating an Extended Task Graph

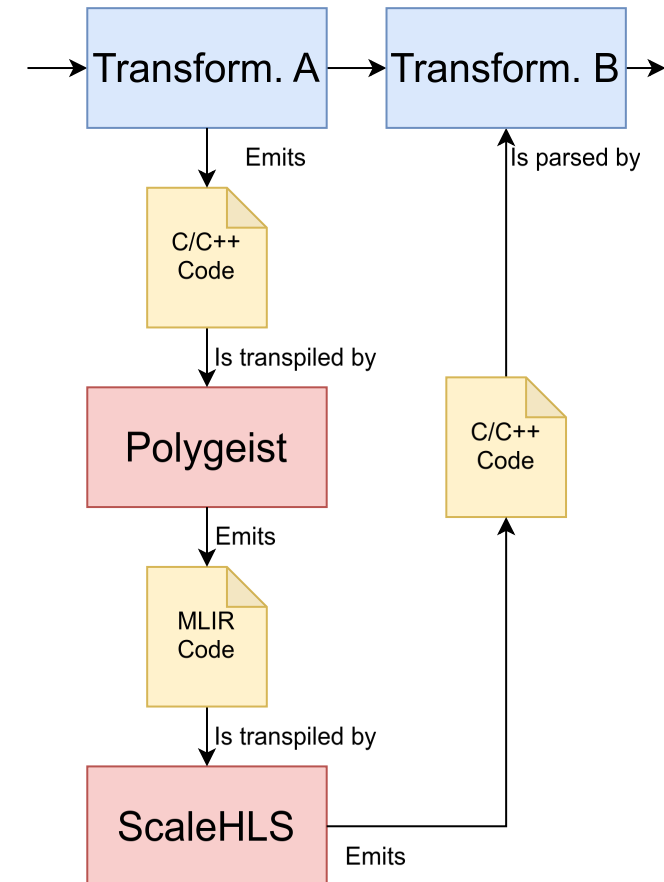
Integration with vendor tools

Vitis HLS

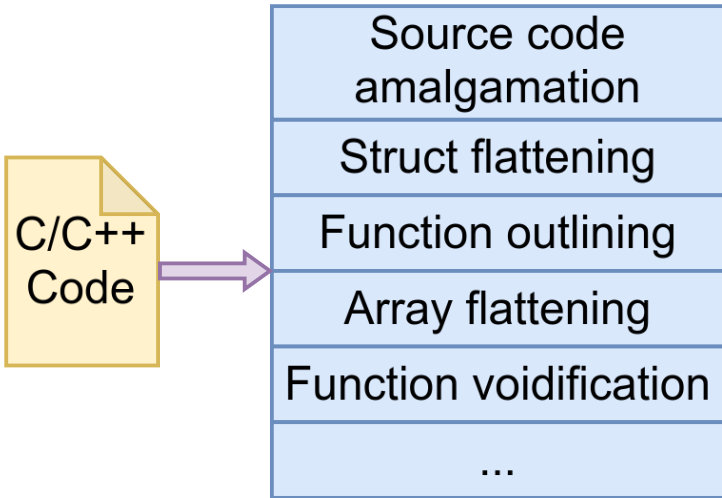


Synthesize any AST function on demand, and build Design Space Exploration flows!

ScaleHLS



Combining all into an HW/SW partitioning flow



Reduce C/C++ entropy,
improve synthesizability

Live demo: Full SW/HW co-design flow

Acknowledgements

My PhD advisors, João M. P. Cardoso and João Bispo

Fundação para a Ciência e a Tecnologia (FCT) for partial funding support under PhD grant 10.54499/2021.07324.BD

And everyone at SPeCS lab who contributed to Clava over the years

Thank you for listening!



The demo is also reproducible, and available here:
<https://github.com/specs-feup/gnu-tools-cauldron-2025-demo>